Goodhope API Documentation

Base URL: ‘http://localhost:4000/api‘

Overview
This backend is a lightweight Node.js HTTP server with MongoDB persistence.
All responses are JSON and include permissive CORS headers.

Common Behavior
- Content type: ‘application/json‘
- Auth: JWT bearer token for all endpoints except ‘/health‘ and ‘/auth/login‘
- Password storage: bcrypt hashes (‘passwordHash‘)
- Compatibility: manual MongoDB Compass users with plain ‘password‘ are accepted once and auto-migrated to ‘passwordHash‘ on successful login
- CORS:
  - ‘Access-Control-Allow-Origin: *‘
  - ‘Access-Control-Allow-Methods: GET,POST,PATCH,DELETE,OPTIONS‘
  - ‘Access-Control-Allow-Headers: Content-Type, Authorization‘
- Invalid JSON payload returns ‘400‘ with:
  - ‘{ "message": "Invalid JSON payload" }‘
- Unknown routes return ‘404‘ with:
  - ‘{ "message": "Not found" }‘

Authentication & Bootstrap

All protected calls must include:

‘‘‘
Authorization: Bearer <jwt>
‘‘‘

GET /health
Health check endpoint.

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

GET /bootstrap
Returns complete bootstrap payload for frontend hydration.

Response 200
‘‘‘json
{
  "packages": [],
  "pilgrims": [],
  "passportDetails": [],
  "payments": [],
  "documents": [],
  "leads": [],
  "users": []

}
```
> ‘users‘ in bootstrap response do not include password.

POST /auth/login
Authenticates user by ‘email‘ and ‘password‘, requires user status ‘Active‘.

**Request body**
```json
{ "email": "admin@goodhope.com", "password": "admin123" }
```

Response 200
```json
{
  "token": "<jwt>",
  "user": {
    "id": "1",
    "name": "Admin User",
    "email": "admin@goodhope.com",
    "role": "SuperAdmin",
    "status": "Active"
  },
  "permissions": ["pilgrims.add", "pilgrims.edit"]
}
```

Response 401
```json
{ "message": "Invalid credentials" }
```

Packages

POST /packages
Creates a package.

Required fields:
- ‘package_name‘
- ‘package_type‘
- ‘makkah_hotel‘
- ‘madinah_hotel‘

‘tags‘ defaults to ‘[]‘.

Response 201: Created package record.

Response 400
```json
{ "message": "Missing required package fields" }
```

PATCH /packages/:id
```

Updates package by ID.
- If ‘tags‘ omitted, existing tags are preserved.

Response 200: Updated package.

Response 404
‘‘‘json
{ "message": "Package not found" }
‘‘‘

DELETE /packages/:id
Deletes package.
- Blocked if package is referenced by any pilgrim.

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

Response 400
‘‘‘json
{ "message": "Cannot delete package assigned to pilgrims" }
‘‘‘

Response 404
‘‘‘json
{ "message": "Package not found" }
‘‘‘

Pilgrims

POST /pilgrims
Creates pilgrim record.
- Auto-generates:
  - ‘id‘ with ‘p‘ prefix
  - ‘registration_no‘ as ‘GH-YYYY-XXX‘
  - ‘created_at‘
  - ‘updated_at‘
- If ‘passport_number‘ provided, creates linked passport detail record.

Response 201: Created pilgrim.

PATCH /pilgrims/:id
Updates pilgrim by ID and refreshes ‘updated_at‘.
- If ‘passport_number‘ present:
  - updates existing passport details, or
  - creates new passport details if none exists.

Response 200: Updated pilgrim.

Response 404
‘‘‘json
{ "message": "Pilgrim not found" }

‘‘‘

DELETE /pilgrims/:id
Deletes pilgrim and cascades related data:
- ‘passportDetails‘
- ‘payments‘
- ‘documents‘

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

Response 404
‘‘‘json
{ "message": "Pilgrim not found" }
‘‘‘

Leads

POST /leads
Creates a new lead.

Required fields:
- ‘full_name‘
- ‘mobile‘
- ‘source‘
- ‘interest‘

Response 201: Created lead.

PATCH /leads/:id
Updates lead record by ID.

Response 200: Updated lead.

Response 404
‘‘‘json
{ "message": "Lead not found" }
‘‘‘

DELETE /leads/:id
Deletes lead by ID.

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

Response 404
‘‘‘json
{ "message": "Lead not found" }

‘‘‘

Payments

POST /payments
Creates payment record.

Response 201: Created payment.

Documents

POST /documents
Creates document record.

Required fields:
- ‘pilgrim_id‘
- ‘name‘

‘doc_type‘ defaults to ‘Other‘.

Response 201: Created document.

Response 400
‘‘‘json
{ "message": "Missing required document fields" }
‘‘‘

DELETE /documents/:id
Deletes document by ID.

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

Response 404
‘‘‘json
{ "message": "Document not found" }
‘‘‘

Users

POST /users
Creates user with defaults:
- ‘password: "welcome123"‘
- ‘status: "Active"‘

Required fields:
- ‘name‘
- ‘email‘
- ‘role‘

Response 201: Created user (without password).

Response 400
```json
{ "message": "Missing required user fields" }
```

Response 409
```json
{ "message": "Email already exists" }
```

PATCH /users/:id/status
Toggles user status Active/Inactive.

Response 200: Updated user (without password).

Response 404
```json
{ "message": "User not found" }
```

PATCH /users/:id/password
Updates user password.
- Minimum length: 4

Response 200
```json
{ "ok": true }
```

Response 400
```json
{ "message": "Password too short" }
```

Response 404
```json
{ "message": "User not found" }
```

PATCH /users/:id
Partially updates user profile fields.

Response 200: Updated user (without password).

Response 404
```json
{ "message": "User not found" }
```

DELETE /users/:id
Deletes user by ID.

Response 200
‘‘‘json
{ "ok": true }
‘‘‘

Response 404
‘‘‘json
{ "message": "User not found" }
‘‘‘

Role Permission Matrix (returned on login)

- ‘SuperAdmin‘:
  - pilgrims.add, pilgrims.edit, pilgrims.delete, pilgrims.view
  - accounts.view, accounts.add, accounts.edit, accounts.delete
  - packages.manage, reports.download, users.manage
- ‘Manager‘:
  - pilgrims.add, pilgrims.edit, pilgrims.view
  - accounts.view, packages.manage, reports.download
- ‘Accountant‘:
  - pilgrims.view
  - accounts.view, accounts.add, accounts.edit
  - reports.download
- ‘DataEntry‘:
  - pilgrims.add, pilgrims.view